Լաբորատոր աշխատանք 10

Դինամիկ անդամներ և լռելայն կոնստրուկտոր

Աշխատանքի նպատակը: Ստեղծել դատարկ և տվյալներով օբյեկտներ, անվտանգ փոխել դինամիկ զանգվածի չափը:

Տեսական մաս: Լռելայն կոնստրուկտորը կանչվում է առանց արգումենտների: Դատարկ օբյեկտում n=0 և data=nullptr արժեքները նկարագրում են վավեր վիճակ: Չափը փոխելիս նախ հատկացնել նոր զանգված, փոխանցել տարրերը և միայն դրանից հետո ազատել հինը:

Աշխատանքի կատարման կարգը

1. Կարդալ ընդհանուր պահանջները և նշանակված անհատական տարբերակը:
2. Սահմանել դասի դաշտերը, դրանց սահմանափակումները և մեթոդները:
3. Իրականացնել պահանջվող գործողությունները և ստուգել սխալ մուտքերը:
4. Համեմատել ստուգիչ օրինակի արդյունքը ձեռքով ստացված հաշվարկի հետ:

Ընդհանուր պահանջներ

1. Դասում պահել private double* զանգված, n չափի և p պարամետր: Լռելայն կոնստրուկտորը ստեղծում է դատարկ օբյեկտ՝ n=0, data=nullptr և պայմանում նշված p-ով: Ավելացնել պարամետրերով կոնստրուկտոր, դեստրուկտոր, append(value), removeAt(index), print() և երեք հաշվարկներ: Չափի փոփոխությունը կատարել նոր զանգված հատկացնելով, տվյալները պատճենելով և հին հիշողությունը ազատելով: Պատճենում և օբյեկտների վերագրում չկիրառել:
2. Տվյալների ծավալ: Սկզբնական ոչ դատարկ հավաքածուի չափը $3 \le n \le 30$ է: Չափի փոփոխություն պահանջող մեթոդներում թույլատրվում է $0 \le n \le 30$: Բոլոր թվերը վերջավոր են: Հավասար լավագույն արդյունքների դեպքում ընտրել առաջին ինդեքսը: Իրական թվերի հավասարությունը համեմատել 10^{-3} ճշգրտությամբ:
3. Դատարկ վիճակ: Գումարները և քանակները զրո են, լավագույն տարրի ինդեքսը՝ -1: Միջինի կամ հարևանների տարբերությունների համար բավարար տարրեր չլինելու դեպքում արտածել «Տվյալները բավարար չեն»: Բազմանդամի դատարկ գործակիցները նշանակում են զրոյական բազմանդամ: Ինտեգրալի համար պահանջվում է առնվազն երկու արժեք:
4. Վավերացում: Սխալ չափով կամ պարամետրերով օբյեկտ չստեղծել. մուտքը կրկնել main()-ում: Թույլատրելիության կանոնները ստուգել նաև փոփոխող մեթոդներում: Սխալ ինդեքսի կամ արժեքի դեպքում վերադարձնել false և պահպանել նախկին տվյալները: Հաշվարկները չպետք է փոխեն օբյեկտը:

Կետերի հեռավորությունը ուղղից

Տվյալներ: Ստեղծել LineDistanceSet դաս՝ պահելով կետերի x կոորդինատների զանգվածը և ընդհանուր y կոորդինատը: Այստեղ $p = y$: Ուղիղը $y = x$ է:

Հաշվարկներ: Հաշվել կետերից ուղղին հեռավորությունների գումարը՝ $d = \vert{}x-y\vert{} / \sqrt{2}$, գտնել ամենահեռու կետի ինդեքսը և $d \le 1$ պայմանով կետերի քանակը:

Թույլատրելի արժեքներ: x և y արժեքները վերջավոր իրական թվեր են:

Իրականացում: Լռելայն $p = 0$: Դատարկ օբյեկտում append-ով ավելացնել օրինակի զանգվածի տարրերը՝ պահելով լռելայն p-ն: Առանձին օբյեկտը ստեղծել օրինակի p-ով:

Առաջինից հեռացնել ինդեքս 1-ի տարրը և երկու օբյեկտների համար հաշվել արդյունքները:

Փորձնական տվյալներ: $x = \{-2, 1, 3, 5\}, y = 1$: Ինդեքսները սկսվում են 0-ից:

```
#include <iostream>
#include <cmath>

using namespace std;

class LineDistanceSet {
private:
    double* d;
    int n;
    double p;

public:
    LineDistanceSet() {
        n = 0;
        d = nullptr;
        p = 0.0;
    }

    LineDistanceSet(const double* arr, int size, double val) {
        if (size < 0 || size > 30) {
            cout << "Սխալ: Չափը պետք է լինի 0-30 սահմաններում:" << endl;
            size = 0;
            d = nullptr;
            p = 0.0;
            return;
        }
        n = size;
        p = val;
        if (n > 0) {
            d = new double[n];
            for (int i = 0; i < n; i++) {
                d[i] = arr[i];
            }
        } else {
            d = nullptr;
        }
    }

    LineDistanceSet(const LineDistanceSet& other) {
        n = other.n;
        p = other.p;
        if (n > 0 && other.d != nullptr) {
            d = new double[n];
            for (int i = 0; i < n; i++) {
                d[i] = other.d[i];
            }
        } else {
            d = nullptr;
        }
    }
```

```
LineDistanceSet& operator=(const LineDistanceSet& other) {
    if (this != &other) {
        delete[] d;
        n = other.n;
        p = other.p;
        if (n > 0 && other.d != nullptr) {
            d = new double[n];
            for (int i = 0; i < n; i++) {
                d[i] = other.d[i];
            }
        } else {
            d = nullptr;
        }
    }
    return *this;
}

~LineDistanceSet() {
    delete[] d;
}

bool append(double val) {
    if (n >= 30) return false;
    int sz = n + 1;
    double* temp = new double[sz];
    for (int i = 0; i < n; i++) {
        temp[i] = d[i];
    }
    temp[n] = val;

    delete[] d;
    d = temp;
    n = sz;
    return true;
}

bool removeAt(int idx) {
    if (idx < 0 || idx >= n || n == 0) return false;
    int sz = n - 1;
    double* temp = nullptr;
    if (sz > 0) {
        temp = new double[sz];
        int k = 0;
        for (int i = 0; i < n; i++) {
            if (i == idx) continue;
            temp[k++] = d[i];
        }
    }
```

```
        delete[] d;
        d = temp;
        n = sz;
        return true;
    }

    void show() const {
        cout << "n = " << n << ", p = " << p << endl;
        cout << "i\tx[i]\ty" << endl;
        cout << "----------------------" << endl;
        for (int i = 0; i < n; i++) {
            cout << i << "\t" << d[i] << "\t" << p << endl;
        }
    }

    void showResult() const {
        if (n == 0 || d == nullptr) {
            cout << "Տվյալները բավարար չեն:" << endl;
            return;
        }

        double sum = 0;
        int max_idx = 0;
        double max_d = -1.0;
        int cnt = 0;
        double s2 = sqrt(2.0);

        for (int i = 0; i < n; i++) {
            double dist = fabs(d[i] - p) / s2;
            sum += dist;

            if (dist > max_d) {
                max_d = dist;
                max_idx = i;
            }
            if (dist <= 1.0) {
                cnt++;
            }
        }

        cout << " Արդյունքներ " << endl;
        cout << "Հեռավորությունների գումար = " << sum << endl;
        cout << "Ամենահեռու կետի ինդեքս = " << max_idx << " (x = " << d[max_idx] << ")" << endl;
        cout << "d <= 1 պայմանին բավարարող քանակ = " << cnt << endl;
    }
};
```

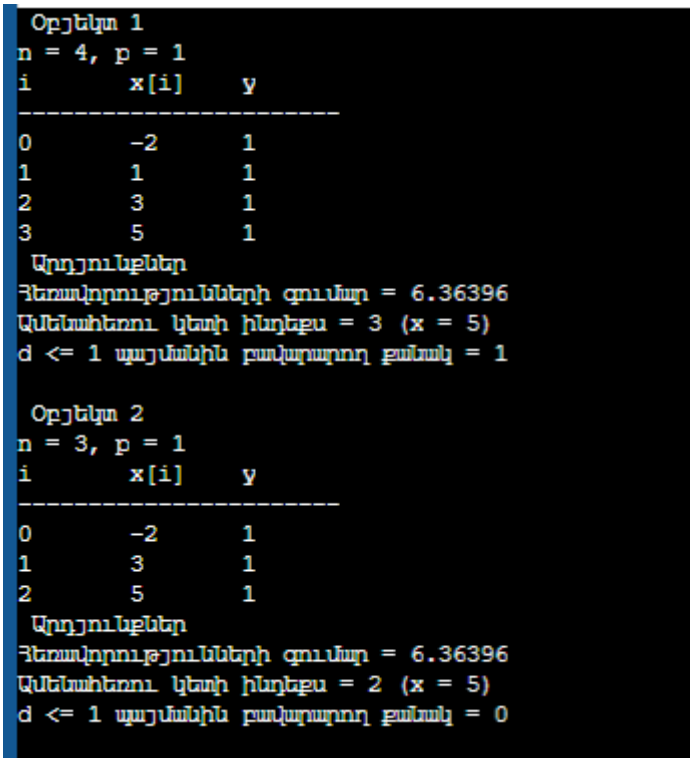
```
int main() {
    double x[] = {-2.0, 1.0, 3.0, 5.0};
    double y = 1.0;

    cout << " Օբյեկտ 1 " << endl;
    LineDistanceSet obj1(x, 4, y);
    obj1.show();
    obj1.showResult();
    cout << "\n";

    cout << " Օբյեկտ 2 " << endl;
    LineDistanceSet obj2 = obj1;
    obj2.removeAt(1);
    obj2.show();
    obj2.showResult();

    return 0;
}
```

Արդյունքը


Օբյեկտ 1
n = 4, p = 1
i x[i] y
0 -2 1
1 1 1
2 3 1
3 5 1
Արդյունքներ
Հեռավորությունների գումար = 6.36396
Ամենահեռու կետի ինդեքս = 3 (x = 5)
d <= 1 պայմանին բավարարող քանակ = 1
Օբյեկտ 2
n = 3, p = 1
i x[i] y
0 -2 1
1 3 1
2 5 1
Արդյունքներ
Հեռավորությունների գումար = 6.36396
Ամենահեռու կետի ինդեքս = 2 (x = 5)
d <= 1 պայմանին բավարարող քանակ = 0